

第 11 章 推荐系统概览

随着大数据与人工智能的发展，我们目前用到的各种软件，包括抖音、快手、微信、美团等等，都接入了推荐系统来进行大规模的内容分发。通俗的讲，推荐系统承载的功能是对软件中的海量信息进行不同用户“千人千面”的个性化推送。根据每个用户在软件中的使用历史记录，推荐系统通过海量数据的学习，可以精确地捕捉每个用户的差异化偏好，从而更好地根据用户兴趣来实现**内容与用户之间的精准匹配**。

对于推荐系统的初学者来讲，一开始对推荐系统的理解可能是一头雾水的，内心也会存在很多疑问。比如：什么是推荐系统？为什么要称之为系统？推荐系统的具体组成部分是怎样的？..... 在本章节，我们会从推荐系统宏观架构的角度出发带读者先从宏观了解推荐系统在工业界实际的系统工程中是如何运作的，接着我们再逐步讲述推荐系统相关的业务目标、在线与离线过程、日志与数据、在线实验设计等方向，最后我们也会详细介绍推荐系统工程上的挑战。在后续的章节中，我们也会逐步深入了解目前推荐系统中最常见的级联架构和该架构下的各个模块以及各个模块中的模型、算法及演进过程。此外，我们也会详细介绍推荐系统工程应用和学术研究中经典的问题以及在大语言模型浪潮下推荐系统领域一些前沿的进展和变化。

首先举一个简单的例子来方便大家理解推荐系统在我们日常使用软件时的作用。以短视频软件为例，通常每天有几亿同时在线的用户，也有至少十亿以上量级可供分发的短视频作品，推荐系统在其中承担着一种桥梁化的作用，如图 11.1 所示。试想一下，如果没有推荐系统，对于几亿的用户，十亿级别的内容，如果两两去做计算和匹配，这将会是用户和内容之间的笛卡尔积，计算开销是非常巨大的。因此，推荐系统对于一款大规模用户量的软件而言有着非常重要的作用，这种作用不仅仅体现在其对于流量分发的精准调控能力，同时也为公司节省了巨大的人力成本，可以更好地提高效率。同时，推荐系统对于以视频或者图文为载体的社区而言，也会间接影响到用户端和内容创作者端，从而长期隐式地去影响整个社区结构、用户偏好、创作者生态、商业化变现等各个方面。

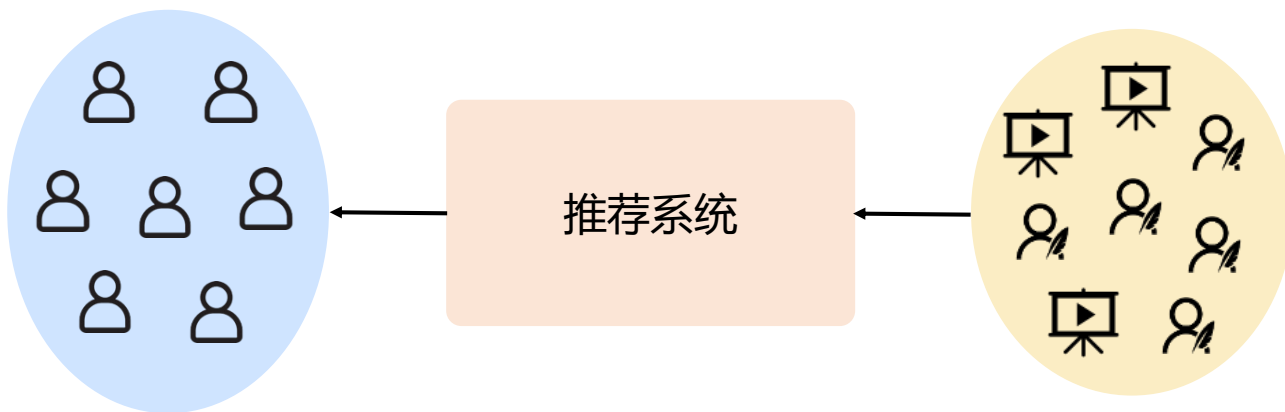


图 11.1: 推荐系统是链接用户和内容（视频/主播）的桥梁。

11.1 系统位置

从图 11.1 中我们知道，**推荐系统是链接用户与内容的桥梁**，解答了推荐系统是什么的疑惑。读者接下来可能会问，那推荐系统在整个一款软件的体系架构中究竟处于什么样的位置，推荐系统又是如何发挥作用的？本章节我们将会从系统架构的宏观视角来揭示推荐系统在整个软件的体系架构中是如何运作的。

具体到用户刷短视频的行为，当用户打开 app 的时候，我们称之为一次会话（session）开始，当用户关闭 app 时，我们称之为这次会话结束。在用户的一次 session 中，随着用户不断往下刷视频，虽然一次性用户能够看几十个视频。但对于整个软件而言，其实并不是一个一个视频向用户进行推送的，而是通过每次用户请求推送多个视频的方式来运行的。这样的一个是好处是，虽然牺牲了一点实时性，但可以大幅降低用户请求对于整个软件系统服务器的调用次数。通常来说，对于用户的一次请求，整个软件系统最终会返回给用户 10 个左右的视

频，当用户快要刷完这 10 个视频的时候，就会在软件系统内自动发起下一次请求，然后系统会返回给用户 10 个新的视频，以此循环运行下去。整个过程如图 11.2 所示。其中，客户端（client）就是手机端，直接与用户进行交互。客户端在接收到用户的请求之后，并不会直接与推荐系统交互，而是先发起服务端（server）的 API 调用。接着，服务端对客户端的请求进行处理和解析之后，也会获取和填充一些新的信息到该请求中，最后才会去调用整个推荐系统。推荐系统经过大量的计算处理之后，最终返回 top 10 的视频给到服务端，服务端进行一些简单的处理再下发到客户端给用户展示。

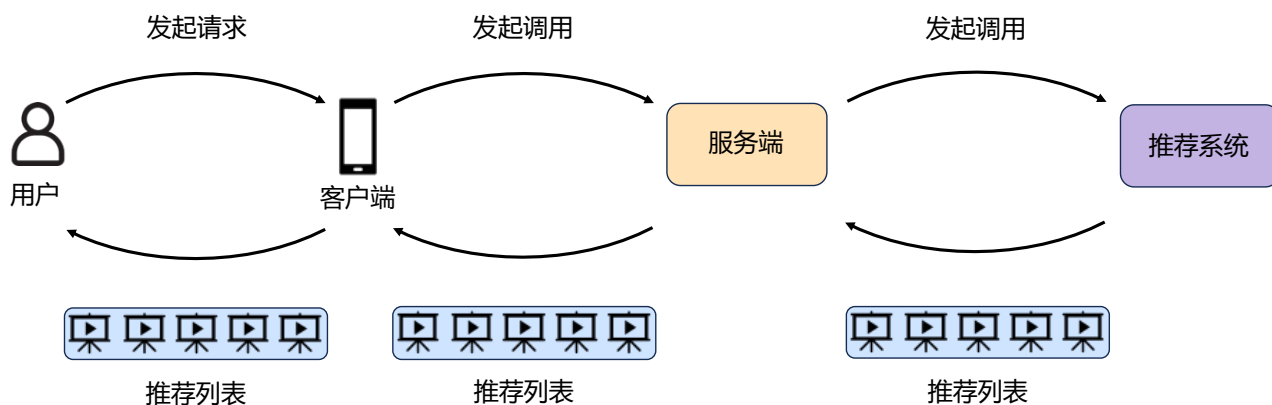


图 11.2: 一次完整的用户请求过程。

需要注意的是，在整个过程中，从用户侧发起这次请求到最终返回推荐结果总的耗时在几百毫秒，这也对整个推荐系统的线上推理速度提出了很高的要求。如果一个推荐算法模型线上推理耗时非常高的话，通常也会拖慢用户请求响应的整个过程，最终让用户感觉这个软件刷起来很卡，这会对用户体验造成非常不好的影响。因此，在推荐领域中，从业人员除了需要去尽可能提升推荐策略以及模型的准确度，也需要考虑在线响应耗时方面的影响。推荐系统的架构师团队会重点关注推荐系统的整体架构以及算力、在线耗时方面的问题，而推荐模型与策略团队会更加关注如何通过推荐算法优化来提升具体的业务指标。

11.2 混合推荐系统

从图 11.2 中可以看到，推荐系统在用户一次请求的调用链路中，并非直接与用户交互，而是处于相对后置的位置。与用户直接交互的是客户端，而客户端与推荐系统之间还隔着一层服务端。服务端不仅承担了信息的转发功能，还包含了大量复杂的业务逻辑。而推荐系统与服务端之间通常通过一个简单的接口层进行对接，这一接口层通常被称为 **路由（router）服务**。

router 服务的作用是将更复杂的推荐系统内部逻辑与服务端解耦隔离。在 router 服务下游，实际存在着面向不同业务方向的多个推荐子系统，它们共同构成了一个复杂的多业务推荐架构，因此我们称之为 **混合推荐系统（Blended Recommendation Systems）**。需要特别指出的是，这里的“混合”并非指推荐算法内部的信号融合，而是指在同一个平台上，多条业务线（如短视频、直播、电商、广告、信息流消息等）所对应的推荐系统在工程架构层面被统一整合在一起。平台级混合推荐的核心关注点，不是单一内容的排序，而是在一次用户请求中，**如何合理分配展示槽位和曝光资源给不同业务**，从而在满足多方目标与约束的前提下实现整体优化。

在这样的混合推荐系统中，系统架构往往相当复杂。以短视频平台为例，平台通常同时包含短视频、广告、电商、直播、图文等多种内容，每一种内容都来自不同的数据源和推荐系统，最终共同组成了一个异质的候选物品列表（heterogeneous item list）。这些推荐结果在汇聚和分发的过程中，体现出混合推荐系统多业务并行的复杂结构。用户的一次推荐请求，从服务端到推荐 router，再到最终的推荐结果返回，整体流程如图 11.3 所示，大致可以分为以下 5 个步骤：

1. 服务端向推荐 router 发起 RPC 请求，请求推荐结果。
2. 推荐 router 接收到请求后，异步并行地向多个不同业务的推荐系统发起调用，分别获取各自的推荐结果。

3. 推荐 router 收集到各业务的推荐结果后，将它们发送给混排服务（blending service）。
4. 混排服务对不同业务返回结果的数量和顺序进行调整，生成最终的混合推荐列表，并返回给推荐 router。
5. 推荐 router 对混排结果进行必要的后处理，然后将最终结果返回给服务端。

需要说明的是，图 11.3 中展示的是图 11.2 中用户请求调用链路的细化版本，用于突出混合推荐系统的关键流程与核心模块。在真实的工业级推荐系统中，混合推荐的架构往往比图中更为复杂，例如还会涉及策略调度、配额分配、竞价机制、多目标优化等环节。为了便于理解，这里只展示了最核心的调用过程和关键组件。

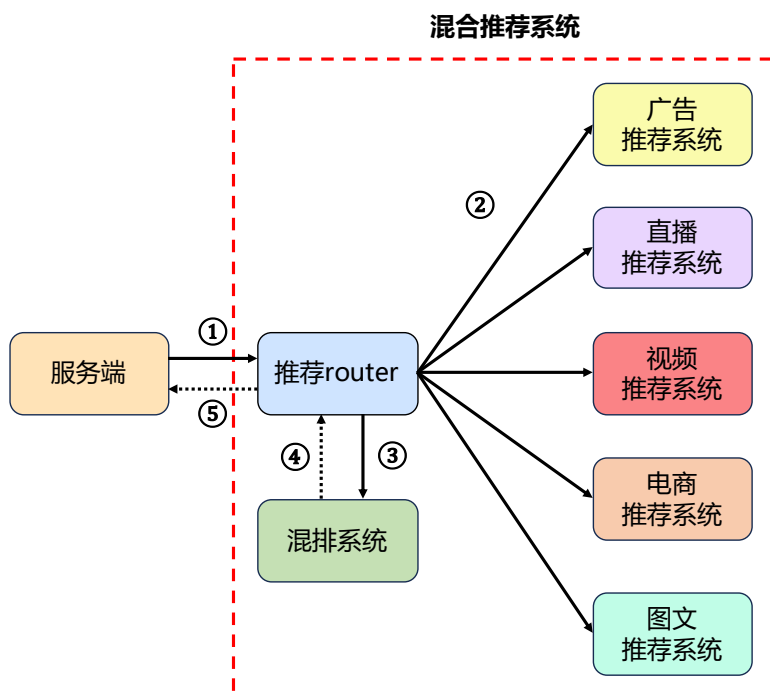


图 11.3: 服务端与混合推荐系统之间的系统架构与调用关系。

在这样的混合推荐系统架构下，现实中的大流量平台通常承载多种业务目标，例如短视频追求用户时长，直播追求互动和打赏，电商追求转化率，广告追求营收。单一业务的推荐优化往往无法满足平台整体目标，因此需要在有限的推荐曝光槽位中平衡不同业务的利益。由此带来的挑战包括：

- **目标冲突**：例如广告曝光可以带来营收，但过多可能损害用户体验。
- **资源有限**：展示槽位稀缺，不同业务都在竞争同一资源池。
- **公平性与约束**：部分业务或创作者可能需要保障最低曝光，避免长期被边缘化。

在工业界的推荐系统优化中，算法工程师除了需要推动自身业务目标的提升，还必须避免陷入“局部最优”，即某一业务单独收益最大化，但却牺牲了平台整体的用户体验或长期指标（如留存、时长）。从数学的角度看，这类问题通常可以抽象为**带约束的全局优化**：不同业务的目标函数在约束条件下被同时考虑，并通过合理的优化方法寻求全局最优解。在实际系统中，这类权衡往往由混排服务来承担：它不仅融合不同业务的推荐结果，还通过引入配额、权重、多目标优化或实时竞价等机制，牵引前链路的混合推荐系统朝着全局最优的方向演化，从而避免系统陷入单一业务驱动的局部最优，保障平台生态的长期发展。

11.3 业务目标

从图 11.3 中我们可以看到，整个混合推荐系统中有广告、直播、视频、电商、图文等多个推荐系统，分别服务于不同的业务。以短视频软件为例，通常视频是整个软件的载体，因此在短视频推荐系统中，我们会重点关注用户的视频播放时长、app 时长、用户留存率、活跃设备数（Daily Active User, DAU）这些核心的业务指标。对于广告业务，我们通常会更加关注广告的总消耗、预期消耗、每次点击成本（Cost Per Click, CPC）、转化率（Conversion Rate, CVR）、千次展示成本（Cost Per Mille / Cost Per Thousand Impressions, CPM）、点击成本（Cost

Per Click)、转化成本 (Cost Per Acquisition) 等业务指标;而直播业务,我们通过会关注直播 DAU、直播时长、直播点击率 (Click Through Rate, CTR) 直播打赏用户数、直播打赏总金额这些业务指标;对于电商业务,我们除了会关注电商视频、电商直播的加购之外,也会关注最终的成交订单总金额 (Gross Merchandise Value, GMV)、千次曝光订单数 (Order Per Mille / Orders Per Thousand Impressions, OPM) 等业务指标。

为了方便大家理解,这里给出 GMV、OPM、CPM、CPC、CPA 这些术语的具体定义和计算公式。其中,GMV 的定义如下:

定义 11.1 (GMV)

GMV 即商品交易总额,是指在一定时间内,通过平台成交的订单总金额 (通常包含已付款 + 未付款但已下单的订单)。具体计算公式为:

$$GMV = \sum (\text{商品单价} \times \text{购买数量}) \quad (11.1)$$

OPM 的定义如下:

定义 11.2 (OPM)

OPM 即千次展示订单数,是指每 1000 次内容 / 商品展示带来的订单数量。具体计算公式为:

$$OPM = \frac{\text{总订单数}}{\text{总展示次数}} \times 1000 \quad (11.2)$$

OPM 与 GMV 的关系可以用如下公式表示:

$$GMV = OPM \times \text{展示次数} \times \text{客单价} / 1000 \quad (11.3)$$

上述两个指标 GMV 和 OPM 是电商推荐业务中非常重要的业务目标。需要注意 GMV 并不是平台最终的收入,平台收入 = GMV × 佣金率 (或广告费)。除此之外,GMV 也包含了退款和取消订单的金额,如果退款率很高的话,GMV 也会存在虚高的问题。对于电商业务来讲,推荐系统需要进行具备精准且极致的流量分发机制,确保能够最大化整个电商业务的成交金额,同时也需要关注流量分发的效率问题,不能搞“大水漫灌”,因此 OPM 指标可以精准地衡量电商推荐中的流量效率,OPM 越大,说明用户看到电商的内容之后更容易下单,转化效率也更高。

而 CPM、CPC、CPA 都是广告场景中的业务目标,具体定义如下:

定义 11.3 (CPM)

CPM 即千次展示成本,是指广告主为每 1000 次广告展示所支付的费用。具体计算公式为:

$$CPM = \frac{\text{广告总花费}}{\text{总展示次数}} \times 1000 \quad (11.4)$$

CPC 的具体定义如下:

定义 11.4 (CPC)

CPC 即每次点击成本,是指广告主为每次用户点击广告所支付的费用。具体计算公式为:

$$CPC = \frac{\text{广告总花费}}{\text{总点击次数}} \quad (11.5)$$

CPA 的定义如下:

定义 11.5 (CPA)

CPA 即每次转化成本,是指广告主为每次有效转化行为 (如下单、注册、提交表单) 所支付的费用。具体计算公式为:

$$CPA = \frac{\text{广告总花费}}{\text{总转化次数}} \quad (11.6)$$

这里需要注意，CPM、CPC 和 CPA 代表了广告领域中几种不同的计费模式，它们的区别不仅在于计算方式，更在于业务目标、风险承担和优化逻辑。它们的具体区别如下表 11.1 所示。

表 11.1: CPM、CPC 与 CPA 的具体区别。

维度	CPM	CPC	CPA
计费基础	曝光	点击	转化
广告主风险	低（展示就付费）	中（点击可能无效）	极低（只为结果付费）
平台风险	低	中	高（需保证转化）
适用广告类型	品牌广告、开屏广告、视频前贴片	搜索广告、信息流广告	效果广告、电商、游戏
是否需要预估模型	否	是（需预估 CTR）	是（需预估 CTR+CVR）
优化目标	提升曝光量、人群覆盖	提升点击率	控制成本、提升 ROI

接下来我们简要介绍一下 CPM、CPC 和 CPA 与广告模型经常需要预估的 CTR 与 CVR 的关系。我们知道：

$$CTR = \frac{\text{点击数}}{\text{曝光数}} \quad (11.7)$$

$$CVR = \frac{\text{转化数}}{\text{点击数}}$$

那么有如下等式成立：

$$CPC = CPA \times CVR$$

$$CPM = CPC \times CTR \times 1000 \quad (11.8)$$

$$CPM = CPA \times CVR \times CTR \times 1000$$

从公式 11.8 我们可以看到，要优化的 CPM 这个广告的业务指标，最终会和广告的 CTR、CVR 这些指标联系起来，这两个指标也是广告算法模型持续去优化的指标。需要强调的是，在工业界的实践中，大家非常关注业务指标的牵引，只有找到一个正确的业务优化指标，才能够借助算法模型、策略的力量来去提升这些业务优化指标。而这些重要的业务优化指标在工业界也通过被称之为“北极星指标”。对于初学者朋友，可能很容易碰到的一个误区是，在对于推荐算法、推荐领域没有特别多认知的情况下，一头扎进具体的算法细节里，而忽略了推荐算法具体要解决什么问题，要优化什么业务指标。

在本小节的讲解中，我们提到了推荐算法在不同细分领域下不同的优化目标，覆盖了传统的短视频推荐、电商推荐、直播推荐以及广告推荐。在短视频推荐中除了传统的视频播放时长的优化目标，大家通常也会去优化整个 App 的 DAU 和用户留存这些指标。而这两个指标也是用户增长算法所要优化的业务目标。由于不同的业务关注不一样的业务目标，所以在推荐算法的实际应用中也会有一些细微的差异，比如广告和电商业务通常在工业界的推荐算法链路中都会去优化 CVR 指标，而在短视频和直播业务中由于业务本身不涉及 CVR，所以通常更关注业务时长、直播打赏等业务目标。读者在学习推荐算法的过程中，除了需要掌握推荐算法的基础知识，也需要去思考不同业务的推荐算法模型所针对的业务细节上的差异。

总的来说，推荐算法在工业界落地应用的时候，通常都把其当做应用算法，即用前沿的算法来解决具体的业务问题。所以对于推荐算法工程师而言，掌握推荐算法的技术细节是基础技能，而具备很强的业务认知，知道业务的痛点是什么，在业务发展的不同阶段制定符合这个阶段的合适的业务目标更为重要。所以读者朋友在学习推荐算法的过程中，一定要牢记这个算法、这个模型乃至这个模块究竟是为了解决什么业务问题，具体优化的业务目标是什么。从这个角度来看，这一小节对于大家未来的学习也会显得非常重要，需要时不时回过头来想一想前面所提到的这些问题点。

11.4 离线与在线

有了前面几节关于推荐系统系统位置、混合结构和业务目标的介绍，这一小节我们具体来了解推荐系统在工业界实际上是如何运作的。其中包括了两个部分，即**离线推荐系统架构**与**在线推荐系统架构**。这两个部分属于推荐系统架构工程师更加关注的内容，但对于未来从事推荐模型与策略工作的读者来说，这一章节可以让大家从一个更加系统化的视角去了解这个问题：即**为什么推荐系统叫做系统**？

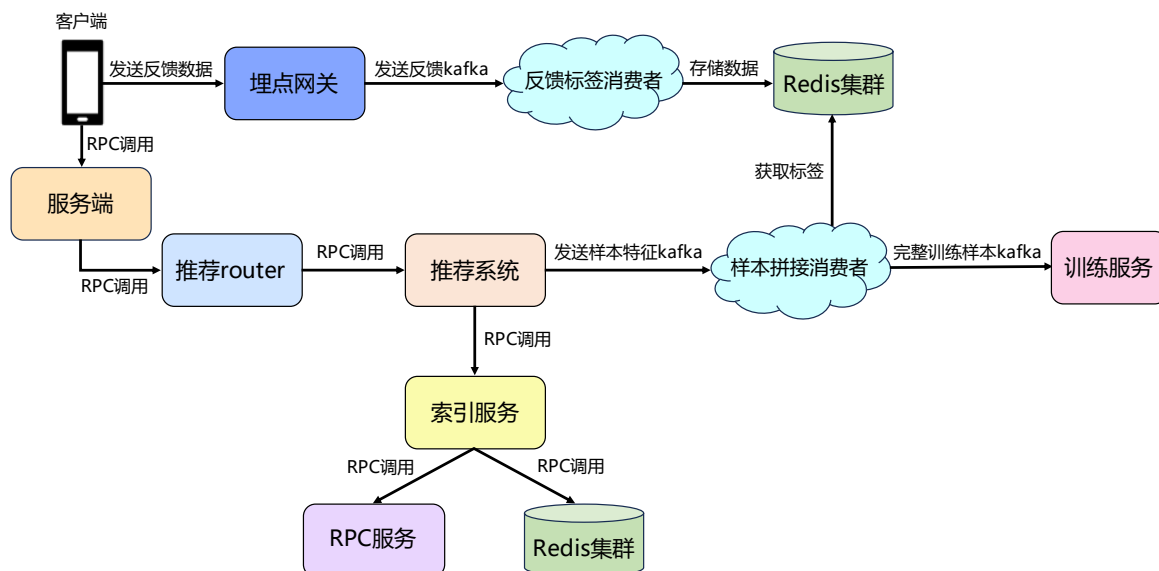


图 11.4: 推荐系统离线架构。

11.4.1 离线架构

推荐系统的离线部分主要包括推荐数据的获取与发送、用户反馈的上传、推荐样本的拼接以及推荐模型的离线训练等环节，如图11.4所示。整个流程包含两个关键分支：

- 当用户与推荐物品发生交互行为（如点击、点赞、分享等）后，客户端会立即将行为日志上报至埋点网关。埋点网关对数据进行鉴权、校验、限流及格式标准化处理后，将其以 **Kafka** 消息的形式发布。下游的“**反馈标签消费者**”（一个基于 **Java** 实现的 **Kafka** 消费服务）会实时消费这些消息，并将用户反馈以 **Key-Value** 的形式写入 **Redis** 缓存。其中，**Key** 通常由 请求 ID + 物品 ID 构成，便于后续样本拼接服务高效查询。
- 在用户每次发起推荐请求时，服务端通过推荐 **router** 调用下游推荐系统。在召回与排序过程中，系统会从索引服务中获取物品的完整特征，并将本次请求的用户特征、物品特征及上下文信息打包为 **Kafka** 消息发送出去。下游的“**样本拼接消费者**”会消费该消息，并在一个固定（例如 20 分钟）或动态设定的拼接窗口内，尝试从 **Redis** 中查询对应的用户反馈。若在窗口期内成功匹配到反馈标签，则将特征与标签拼接为完整的训练样本 (*user, item, context, label*)，并写入新的 **Kafka Topic**，供离线模型训练任务消费。

以短视频场景为例，如图11.5所示，用户在观看视频时可进行点赞、评论、分享等交互操作。当用户完成点赞或分享后，客户端会将这些行为以 **JSON** 格式上报至埋点网关。埋点网关处理后写入 **Kafka**，由“反馈标签消费者”写入 **Redis**；而“样本拼接消费者”则通过等待机制尝试拼接正样本，其核心目标是**最大化样本拼接率**，从而提升模型训练的数据质量与效果。

在图11.4所示的推荐系统离线架构中，涉及若干关键技术组件，此处统一说明如下：

- **Kafka**: **Kafka** 是一个高吞吐、分布式的流式消息队列系统，广泛应用于日志收集、数据流传输和解耦不同系统之间的数据交互。在推荐系统中，**Kafka** 主要用于实时收集用户行为日志（如曝光、点击、停留时长等），并将这些数据准实时地传输至下游的特征工程、样本拼接或模型训练模块。

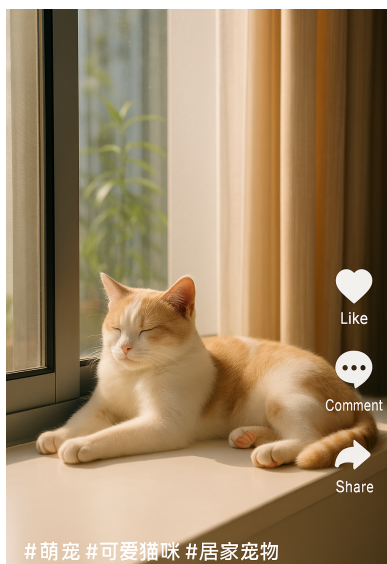


图 11.5: 短视频页面示意图。

- **索引服务:** 此处的索引服务特指物品侧 (Item-side) 的特征索引服务, 即根据物品 ID 快速检索其完整特征 (如 embedding 向量、属性标签、统计特征等)。该服务在推荐系统的线上推荐排序时被调用, 解决“已知 item ID 如何高效获取特征”的问题。通常我们也将之称为正排索引。需注意, 它与召回阶段使用的倒排索引或向量索引不同: 后者面向大规模候选集的快速检索, 而前者面向单点特征的高效查询, 二者功能互补。
- **RPC 调用:** RPC (Remote Procedure Call, 远程过程调用) 是一种跨服务通信机制, 允许调用方像调用本地函数一样访问远程服务。在推荐系统中, 特征服务、用户画像服务、模型在线推理服务等模块通常通过 RPC 进行交互, 以满足低延迟、高并发的在线推理需求。
- **Redis 集群:** Redis 是一种基于内存的高性能键值存储系统, 支持极低延迟的读写操作。Redis 集群通过分片 (sharding) 和副本 (replication) 机制实现水平扩展与高可用性。在推荐系统中, Redis 常用于缓存用户特征、物品特征、中间计算结果或召回结果等, 可以显著加速在线推理与离线拼接流程。
- **样本拼接服务:** 样本拼接服务负责将来自不同源头的异构数据 (如用户行为日志、物品特征、用户画像、上下文信息等) 融合为结构化的训练样本, 并输出为 (user, item, context, label) 的格式, 供监督学习模型训练与评估使用。该服务是推荐系统离线训练流水线中的关键环节, 直接影响样本质量与模型效果。

读者可能会注意到, 图 11.4 中离线训练所依赖的数据源是实时 Kafka 消息流, 而非传统推荐学习中常见的静态用户-物品交互矩阵。这一设计源于工业级推荐系统的现实约束: 在短视频、电商等场景下, 每日活跃用户数与内容创作者产出的物品规模极其庞大, 难以以稠密矩阵形式显式存储全部交互数据。此外, 为更及时地捕捉用户兴趣的动态变化、提升推荐的准确性与实时性, 系统倾向于采用基于实时流的训练范式。当然, 并非所有场景都必须使用流式训练。例如在广告推荐中, 若用户单日接触的广告数量有限, 也可采用基于 HDFS 等离线存储的批处理方式, 按小时或数小时粒度定期训练模型。推荐算法工程师需根据业务特性 (如数据规模、反馈延迟、实时性要求等), 在 **实时流训练** 与 **离线批训练** 之间进行权衡与选择。

11.4.2 在线架构

推荐系统的在线部分相比离线部分简单一些, 在前几个章节中我们也有所涉及。相比离线部分, 在线推荐系统更注重 **低延迟、高可用与稳定性**。它是客户端 → 服务端 → 推荐 router → 推荐系统主服务这一整个链路中的后半部分, 主要负责了根据用户的一次请求, 从物品的海量候选池中去筛选出用户最可能感兴趣的 top K 个物品。与推荐系统离线部分不同的是, 在线系统通常直接承载用户的请求, 需要更快的响应速度和更高的稳定性。如果推荐服务出现不可用的情况, 通常可能出现推荐结果为空, 在用户体验的时候可能会发现整个 app 刷不动

或者刷着很卡顿、刷不出东西等现象。这时候，在实际推荐系统在线部分的设计中，会加入一些偏规则的“兜底机制” (backup mechanism)，比如推荐系统服务挂掉的时候，可以从高热的物品候选池中随机选取物品推荐给用户，这样虽然会降低推荐的准确度，但能保障推荐系统的基本可用性，也不太会对用户的软件使用体验产生较大的负面影响。

除此之外，推荐系统在线架构的设计中，也会根据用户使用软件的高低峰期情况去进行**动态算力分配**。如图11.6所示，是推荐系统高低峰期每秒钟调用次数 (Query Per Second, QPS) 的示意图，可以看出来明显的波峰和波谷的形态。对于波峰形态时，对应着大量用户集中使用软件的时间段；而对于波谷形态时，对应着少部分用户集中使用软件的时间段。在高峰期的时候，通常对于工业界推荐系统的算力消耗是非常大的，这时候为了平衡推荐的精准度和核心的业务指标（比如用户 APP 使用总时长、用户长期留存等）与算力成本消耗，因此工业界通常在常规的实时推荐系统之外，还有一套**降级推荐系统**，即使用少量的算力资源来达到略差于实时推荐效果的推荐系统，可以节省大量的服务器算力成本。

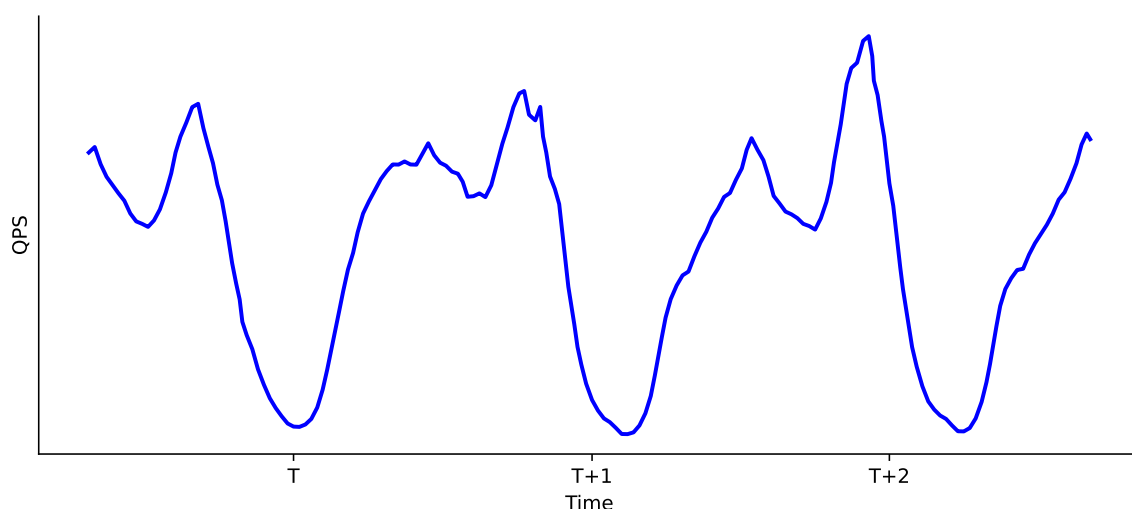


图 11.6: 推荐系统 QPS 高低峰期示意图。

一种典型的降级策略是：在实时推荐系统返回结果时，将 Top 5 至 Top 20 的物品列表缓存至 Redis。当系统进入降级状态时，直接从 Redis 读取预存结果返回，避免执行完整的召回-排序流程。降级与实时系统的协同架构如图11.7所示，其决策流程如下：

1. 在用户的一次请求中，推荐 router 先通过 RPC 调用请求降级决策服务。
2. 降级决策服务返回给推荐 router 是否降级的 Flag 字段，字段取值为 bool 类型，即 0 或者 1。
3. 推荐 router 根据是否降级的 Flag 字段取值来请求实时推荐系统或降级推荐系统。如果 Flag=0，则请求实时推荐系统；反之如果 Flag=1，则请求降级推荐系统。

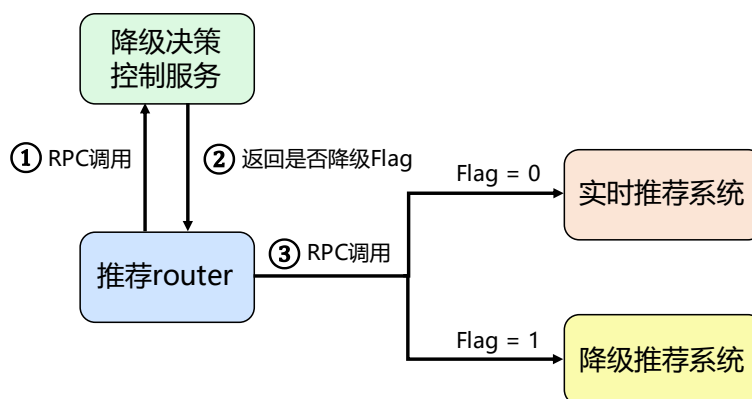


图 11.7: 降级与实时推荐系统架构图。

注意在图11.7中，降级决策服务中所使用的降级决策控制算法可以通过简单策略来控制，也可以通过强化学习来控制。在该场景下，使用强化学习进行是否降级的流量控制我们放到后续章节17中进行介绍。在本节中，我们介绍一种叫做 DCAF [1] 的算法。DCAF (Dynamic Computation Allocation Framework) 源自论文《DCAF: A Dynamic Computation Allocation Framework for Online Serving System》，其核心思想是：打破传统系统对所有请求“一视同仁”的算力分配方式，转而根据每个请求的潜在商业价值动态分配计算资源。

在典型的工业级推荐或广告系统中，服务架构通常采用多阶段级联设计（如召回、粗排、精排等），每个阶段往往被分配固定的计算配额（例如精排阶段最多评估 200 个候选）。这种静态配额策略忽略了不同用户请求之间巨大的价值差异——高价值用户（如高活跃、高消费倾向）理应获得更精细的模型计算，而低价值请求则可适当降级以节省资源。DCAF 正是为解决这一问题而提出的动态算力分配框架。

形式上，DCAF 将该问题建模为一个带约束的背包问题：在总计算资源预算 C 下，为每个请求 i 从 M 个可选动作（对应不同算力开销 q_j 和预期收益 Q_{ij} ）中选择一个，以最大化系统总收益：

$$\begin{aligned} \max_{x_{ij}} \quad & \sum_{i=1}^N \sum_{j=1}^M x_{ij} Q_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^N \sum_{j=1}^M x_{ij} q_j \leq C, \\ & \sum_{j=1}^M x_{ij} \leq 1, \quad x_{ij} \in \{0, 1\}, \end{aligned} \quad (11.9)$$

其中 $x_{ij} = 1$ 表示为请求 i 分配动作 j ， Q_{ij} 通常定义为该请求在动作 j 下的预估 eCPM（有效千次展示收益）， q_j 则对应动作 j 的计算开销（如需精排的广告数量）。

通过拉格朗日对偶分析，DCAF 证明：在合理假设下（如收益随算力单调递增但边际收益递减），最优策略具有简洁形式——对每个请求 i ，选择使 $Q_{ij} - \lambda q_j$ 最大的动作 j ，其中 λ 是与全局资源预算 C 相关的拉格朗日乘子。该 λ 可通过高效的二分搜索离线求解，确保在满足总预算约束的同时最大化系统总收益。

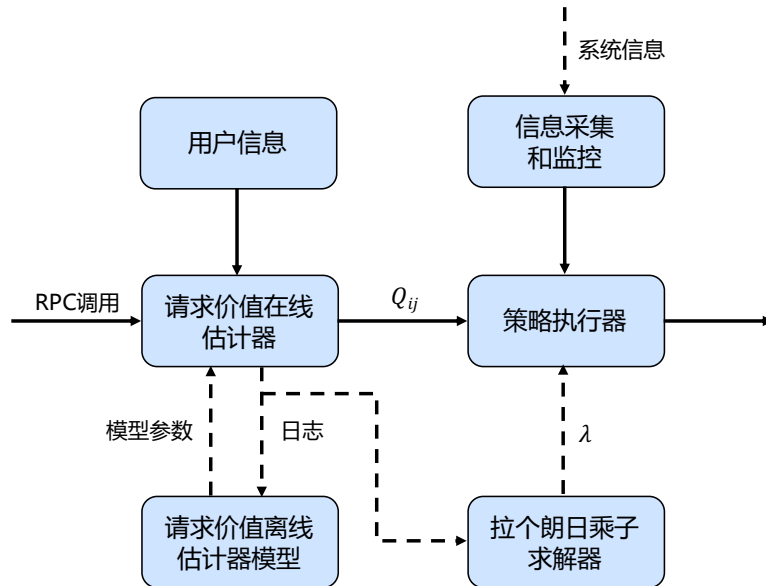


图 11.8: DCAF 系统架构示意图。

在线服务调用时，DCAF 系统由两大模块构成（如图11.8 所示）：

- **请求价值估计器 (Offline Estimator)**：基于历史日志训练轻量级模型，实时预估每个请求在不同动作 j 下的 Q_{ij} （通常通过预估 CTR × 广告出价得到 eCPM）。为避免增加线上负担，该模型需兼顾效率与精度，并可复用已有模块的中间特征。
- **策略执行器 (Online Decision Maker)**：结合预估的 Q_{ij} 与当前最优 λ 值，为每个请求动态选择动作 j 。此

外，为应对流量突刺（如双 11 大促），DCAF 引入了 **MaxPower** 机制——通过 PID 控制器实时监控系统状态（如 GPU 利用率、失败率、响应延迟），动态限制单请求最大算力开销 q_j ，从而保障系统稳定性。

DCAF 已在淘宝展示广告系统中全量上线。实验表明，在保持广告收入（RPM）和点击率（CTR）基本不变的前提下，DCAF 可将精排阶段的 GPU 资源消耗降低约 20%，同时显著提升系统在高并发场景下的鲁棒性。与传统的全局降级策略相比，DCAF 实现了“千人千面”的算力分配，在资源受限时优先保障高价值流量的体验，从而在业务指标与成本之间取得更优平衡。

最后我们来简要介绍降级推荐系统内部的主要部分，如下图 11.9 所示。降级推荐系统内部的处理流程包括如下几个步骤：

1. **降级 Redis 缓存召回**: 在实时推荐系统中每次请求都会将最新的排序结果以每个用户个性化的方式写入降级 Redis 缓存中。当该用户的降级请求来临时，会从降级 Redis 缓存中召回和检索最新的排序结果。
2. **过滤规则**: 根据用户的观看历史，对重复推荐的内容进行过滤，防止伤害用户体验。同时，也有一些其他业务相关的过滤逻辑存在，比如风控安全向的过滤规则等等。
3. **截断**: 由于实时推荐系统在缓存降级推荐结果的时候，通常会缓存比较多的物品，所以一次降级推荐使用不完的话，就对 top K 的待推荐物品列表进行截断，后面的物品列表留作后续的降级推荐请求时使用。

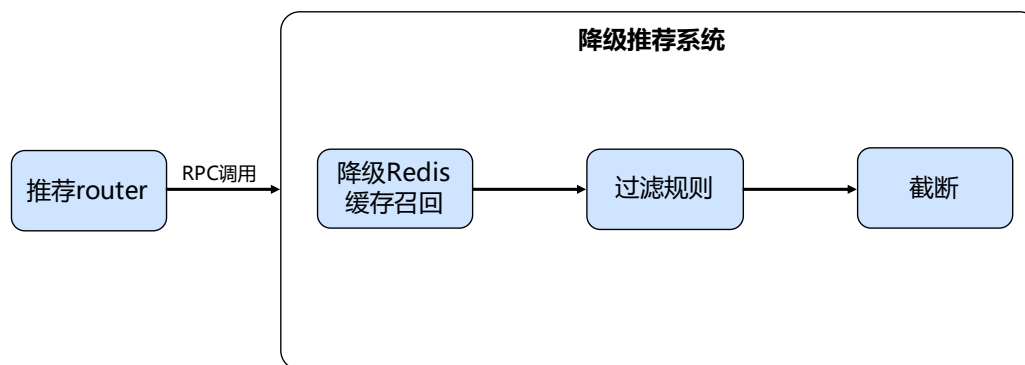


图 11.9: 降级推荐系统内部结构示意图。

11.5 日志与数据

推荐系统中的日志与数据是提升推荐算法模型精度的核心基础。尽管我们常将这些推荐算法模型泛称为“AI 模型”，但归根结底，它们本质上是**高度依赖数据驱动**的统计学习系统。模型本身并不具备先验知识，其对用户兴趣的理解、对物品价值的判断，完全建立在历史用户与物品的交互行为之上。若缺乏高质量的用户-物品交互数据，模型便无法有效捕捉用户复杂的兴趣偏好，推荐效果自然无从谈起。

与早期基于用户-物品打分矩阵（如 **MovieLens** 数据集）的传统协同过滤方法不同，现代工业级推荐系统所面对的数据环境要复杂得多。一方面，系统可利用的信息维度极大丰富：不仅包括用户的历史行为序列（如点击、观看时长、点赞、分享、收藏等），还融合了**用户画像**（如年龄、地域、设备、活跃时段）、**物品特征**（如类别、标签、Embedding、内容理解结果）以及**上下文环境**（如时间、地点、网络状态、当前会话路径）等多模态特征。另一方面，用户的兴趣并非静态不变，而是呈现出**动态演化特性**——可能因短期热点而产生兴趣增强，因内容重复而发生兴趣衰减，或因生活阶段变化而出现兴趣转移。这些复杂的行为模式对推荐模型的表达能力与学习机制提出了更高要求。

与此同时，工业场景下的日志数据规模通常达到 **TB 甚至 PB 级别**，每日新增的用户请求与反馈事件数以亿计。如此庞大的数据量虽为推荐模型训练提供了充足“燃料”，但也带来了严峻挑战：

- **数据质量参差不齐**: 日志中可能混杂因客户端异常、网络抖动、埋点错误或恶意刷量产生的**噪声数据**或**错误样本**；
- **反馈稀疏与延迟**: 用户仅对极少数曝光物品产生显式反馈，且部分反馈（如转化、分享）存在显著延迟；
- **分布偏移**: 训练数据与线上真实分布可能存在偏差（如曝光偏差、选择偏差），导致模型泛化能力下降。

因此，如何从海量、高维、稀疏且含噪的日志数据中**高效提取有效信号**，构建鲁棒、公平且具备长期价值的推荐模型，已成为工业界推荐系统研发中的核心课题。这也促使推荐算法不断从单纯的“拟合历史行为”向“理解用户意图”、“建模长期兴趣”乃至“探索与利用平衡”等更深层次演进。

在这一背景下，推荐系统所依赖的数据通常以两种主要形式存在：**实时数据流**与**离线批量数据**。从前文介绍的离线架构可知，样本拼接服务通过 Kafka 等消息队列实时消费用户请求与反馈日志，生成用于模型训练的流式样本。这类数据流的核心目标是支撑**近线**（nearline）与**在线**（online）模型更新，从而快速响应用户兴趣的动态变化。需要说明的是，“**近线**”更新通常指利用实时行为流增量计算并刷新用户或物品的动态特征（如最近点击序列、短期兴趣向量等），而模型结构本身仍沿用离线训练的结果；“**在线**”更新则更进一步，指**模型参数本身在收到新反馈后立即进行学习**与调整（如通过在线学习算法）。在实际工业系统中，二者常协同使用：近线特征提供上下文感知能力，在线学习则实现对反馈的即时响应。另一方面，系统还会将原始日志、特征快照、拼接后的训练样本等数据定期归档至**离线数据仓库**，用于回溯分析、特征有效性验证、AB 实验评估以及大规模离线模型训练。这类批量数据虽存在时效延迟，但覆盖全面、质量可控，是模型迭代与策略验证的重要基础。

在工业实践中，这类离线数据通常以 Hive 表的形式组织存储。Apache Hive 是构建在 Hadoop 之上的数据仓库基础设施，它提供了一种类 SQL 的查询语言，Hive SQL（也称 HQL），使得开发者无需编写复杂的 MapReduce 程序，即可对存储在 HDFS（Hadoop 分布式文件系统）上的海量结构化或半结构化数据进行高效查询、聚合与分析。例如，算法工程师可通过 Hive SQL 快速统计某类用户在特定时间段内的点击率、分析特征覆盖率、验证新特征的有效性，或抽取用于离线训练的全量样本集。因此，掌握 Hive SQL 已成为推荐系统从业者的一项基本技能。无论是进行日常的数据探查、特征验证，还是构建离线训练 pipeline，Hive 都扮演着连接原始日志与算法模型之间的重要桥梁角色。

在掌握了 Hive SQL 这一关键工具后，算法工程师便可对推荐系统中的原始日志进行深入分析与特征提取。以短视频推荐场景为例，系统通常会围绕用户、内容与交互三大维度，沉淀出结构清晰、字段丰富的日志体系。这些日志不仅是离线特征计算和样本构建的基础，也是理解用户行为模式、评估推荐效果的核心依据。具体而言，关键日志主要包括以下几类：

- **用户侧基础信息日志**：记录用户的静态画像与设备属性，通常每日快照一次，用于刻画用户的长期兴趣背景与行为环境。字段包括：用户的用户 ID、设备 ID、年龄、性别、兴趣偏好、机型、手机系统、手机 App 版本、是否新用户、是否登录等。
- **短视频基础信息日志**：描述视频内容及其元数据，由内容理解系统或创作者后台生成，通常在视频发布时写入，并随互动数据动态更新。字段包括：短视频的视频 ID、视频内容时长、视频创作者用户 ID、视频创作者设备 ID、视频话题 tag、视频点赞数量、视频评论数量、视频分享数量、视频转发数量等。
- **用户-短视频交互信息日志**：记录用户与视频在不同时间粒度下的行为，是推荐模型训练的核心信号来源，分为两类：
 - **请求粒度的信息日志**：每次推荐请求生成一条记录，包含曝光上下文与即时反馈，用于构建实时训练样本。字段包括：用户与短视频交互的请求 ID、用户 ID、设备 ID、短视频的视频 ID、视频创作者用户 ID、视频创作者设备 ID、视频观看时长、是否点赞、是否评论、是否分享、是否转发、是否观看小于 3s（视频短播）、是否观看大于 6s（视频有效播放）、是否看完整个视频（视频完播）等；
 - **用户与短视频天级交互日志**：按用户-视频对每日汇总行为统计，用于长期兴趣建模与特征回溯。字段通常包括：用户与短视频交互的请求 ID、用户 ID、设备 ID、短视频的视频 ID、视频创作者用户 ID、视频创作者设备 ID、天级累计视频观看时长、天级累计视频点赞数量、天级累计视频评论数量、天级累计视频分享数量、天级累计视频转发数量等。

为了帮助读者更直观地理解上述日志的数据形态，我们基于实际业务场景模拟了三张简化示例表（数据为虚构）：

表 11.2 展示了用户侧基础信息日志的典型结构。该表通常以天为单位进行快照更新，用于构建用户的静态画像。其中，user_id 和 device_id 是跨会话行为归因的关键标识；age 与 gender 提供基础人口统计特征，常用于分群策略或特征交叉；interest_tags 由用户历史行为聚类或内容偏好模型生成，例如“# 美食,# 旅行”反映了用户对生活类内容的长期兴趣；os 与 app_version 可用于分析不同设备或版本下的用户行为差异；而 is_new_user 字段

表 11.2: 用户侧基础信息日志示例

user_id	device_id	age	gender	interest_tags	os	app_version	is_new_user
10086	d7a3b2c1	28	female	# 美食,# 旅行	Android 12	v3.4.1	false
20240	e9f8d7c6	19	male	# 游戏,# 科技	iOS 16	v3.4.0	true
30571	a1b2c3d4	35	male	# 财经,# 新闻,# 汽车	Android 13	v3.4.2	false
40892	f5e4d3c2	24	female	# 美妆,# 穿搭,# 健身	iOS 17	v3.4.1	false
51203	9z8y7x6w	42	female	# 教育,# 亲子,# 家居	Android 11	v3.3.9	true
62314	m2n3o4p5	17	male	# 动漫,# 电竞,# 短视频	iOS 15	v3.4.0	true

则直接服务于冷启动逻辑，如第二行所示，用户 20240 为新用户（is_new_user = true），系统可能为其启用探索性推荐策略或默认兴趣兜底。

表 11.3: 短视频基础信息日志示例

video_id	author_id	author_device_id	duration_sec	tags	like_cnt	comment_cnt	share_cnt
v12345	70349	z5f8c5k0	45	# 萌宠,# 搞笑	12500	842	310
v67890	81426	q2w9e7r4	28	# 旅行,# 海岛	3200	156	98
v24680	93571	m3n8b2v6	62	# 知识,# 科普	8900	1203	742
v13579	64802	t7y4u9i1	18	# 美妆,# 教程	21000	2450	1830
v97531	55128	j6h3g9f2	55	# 剧情,# 情感	45600	3890	2100
v86420	47293	x8c4v6b1	12	# 音乐,# 卡点	6700	320	410

表 11.3 描述了短视频的基础元数据。视频在发布时即写入初始信息，并随用户互动行为准实时更新计数类字段。video_id 作为内容唯一标识，是特征索引与样本拼接的核心键；author_id 与 author_device_id 用于关联创作者画像；duration_sec 不仅决定视频长度，还影响完播率等关键指标的计算；tags 通常由人工打标、自然语言处理或计算机视觉模型自动生成，如“# 萌宠,# 搞笑”可用于语义召回通道；而 like_cnt、comment_cnt、share_cnt 等互动统计数据，既是内容热度的直接体现，也可作为排序模型中的统计特征（例如经平滑处理后的分享率）。例如，视频 v12345 虽仅 45 秒，但已积累上万点赞，表明其具有较高用户吸引力。

表 11.4: 用户-短视频交互日志示例（请求粒度）

req_id	user_id	device_id	video_id	author_id	author_device_id	watch_sec	is_like	is_comment
r98765	10086	d7a3b2c1	v12345	70349	z5f8c5k0	42	true	false
r87654	20240	e9f8d7c6	v67890	81426	q2w9e7r4	28	false	true
r76543	30571	a1b2c3d4	v24680	93571	m3n8b2v6	2	false	false
r65432	40892	f5e4d3c2	v13579	64802	t7y4u9i1	18	true	true
r54321	51203	9z8y7x6w	v97531	55128	j6h3g9f2	55	true	false
r43210	62314	m2n3o4p5	v86420	47293	x8c4v6b1	5	false	false

表 11.4 展示了请求粒度的用户-短视频交互日志，这是推荐系统中最核心的“曝光-反馈”数据。每条记录对应一次推荐请求，包含完整的上下文与多维反馈信号。req_id 唯一标识一次请求，是样本拼接的基础；watch_sec 表示实际观看时长，结合视频总时长可衍生出多种行为标签：例如，若观看时长小于 3 秒，则 is_short_watch = true，通常视为无效曝光（负样本）；若大于 6 秒，则 is_valid_play = true，作为弱正样本信号；若观看时长等于或接近视频总时长，则 is_complete = true，反映内容具有强吸引力。此外，is_like、is_comment、is_share 等显式反馈行为共同构成多目标学习的标签体系。例如，第一行中用户 10086 观看视频 v12345 总共 42 秒（未完播），但进行了点赞和分享，表明其对内容高度认可；第二行中用户 20240 完整观看了 28 秒的视频 v67890 并评论，说明内容与其兴趣高度匹配。这些细粒度信号为精排模型的多任务学习提供了丰富监督信息。

在实际工程中，上述三类日志均以 Hive 表形式存储于离线数据仓库。通过 Hive SQL 的 JOIN 操作，可将请求日志与用户画像表、视频信息表关联，从而构建出包含用户特征、物品特征、上下文特征及多维反馈标签的完整训练样本 (user, item, context, label)，为推荐模型的迭代优化提供坚实数据基础。然而，传统视频元数据（如时长、原始标签等）往往语义稀疏，难以充分刻画内容的深层主题与情感倾向。为提升物品侧特征的表达能力，

我们进一步引入大语言模型（Large Language Models, LLMs）与多模态大模型（Vision-Language Models, VLMs）对视频内容进行深度理解，并将模型输出结构化为增强型元数据，融入样本构建流程。

具体而言，在短视频场景中，LLM 可对视频的语音识别文本（ASR）进行语义解析，自动生成结构化的内容描述、核心话题与垂类标签（如“萌宠日常”“硬核科普”“旅行攻略”等），有效弥补人工标签覆盖有限的问题；同时，LLM 还能聚合分析评论区的高互动评论，提炼出具有代表性的用户观点摘要与情感倾向，形成对内容价值的群体共识信号。与此同时，VLM 通过联合建模视频的视觉帧、音频与文本信息，输出统一的多模态嵌入向量（visual_emb），该向量在语义空间中实现了跨模态对齐，可作为高质量的内容表征用于召回或排序阶段。

为支持上述能力的工程落地，我们构建了一张 LLM/VLM 增强的短视频元数据表（见表 11.5）。该表以 video_id 为主键，并新增由模型生成的 asr_summary、topic、category、comment_summary 与 visual_emb 字段。其中，asr_summary 是 LLM 对视频语音转文字内容生成的简洁摘要，topic 表示核心讨论主题，category 为模型判定的垂类标签，comment_summary 是对高赞评论的语义聚合结果，visual_emb 则为 VLM 输出的稠密向量（通常为 768 维），在表中仅展示前几位作为示意。该表可与用户交互日志、用户画像等表通过 video_id 高效关联，显著增强训练样本中“item”与“context”的语义丰富度。

表 11.5: LLM/VLM 增强的短视频元数据表示例

video_id	asr_summary	topic	category	comment_summary	visual_emb
v12345	一只橘猫被逗猫棒逗得疯狂扑跳，全程搞笑不断。	萌宠互动	宠物	“猫咪反应太可爱了！笑到肚子疼”	[0.82, -0.31, 0.45, ...]
v67890	探访马尔代夫私人岛屿，展示日落海滩与水上别墅。	海岛旅行	旅行	“想去这个地方！摄影太美了”	[-0.12, 0.67, -0.23, ...]
v24680	用生活实例解释牛顿三大定律，通俗易懂。	物理科普	知识	“讲得通俗易懂，期待下一期”	[0.55, 0.28, 0.91, ...]
v13579	演示新手如何画出自然日常眼妆，步骤清晰。	美妆教程	美妆	“步骤清晰，已收藏反复看”	[0.33, -0.44, 0.76, ...]
v97531	讲述单亲父亲独自抚养女儿的温情故事。	亲情	剧情	“看哭了，真实得让人心疼”	[-0.61, 0.19, -0.52, ...]
v86420	卡点舞蹈配合热门BGM，节奏感极强。	舞蹈卡点	音乐	“节奏感绝了！BGM 求链接”	[0.70, 0.88, -0.15, ...]

通过将此类由 LLM 与 VLM 生成的语义增强数据与用户行为日志深度融合，推荐系统的能力边界得以显著拓展：不仅能够基于历史交互“知道用户喜欢什么”，更能借助深层次的内容理解“洞察内容为何被喜欢”。这种从行为信号驱动向语义理解驱动的范式跃迁，使得推荐系统在捕捉用户兴趣、识别内容价值以及建模用户-内容匹配关系等方面更加精准、鲁棒且具备可解释性。例如，当模型理解到某视频被喜爱是因为其“通俗易懂的物理讲解”而非仅因“高完播率”，便能在相似语义场景下实现更泛化、更合理的推荐。上述由大模型赋能的内容理解能力，已逐步成为新一代智能推荐系统的核心基础设施。本文后续章节将系统性地展开讨论：包括 LLM 如何处理 ASR 文本与用户评论以构建结构化语义标签，VLM 如何生成高质量的多模态表征并应用于召回与排序阶段，以及如何将这些大模型输出有效融入多任务学习框架，实现语义信号与行为信号的协同优化。

11.6 在线实验设计

在工业界推荐系统的算法迭代与产品优化中，线上 A/B 测试（A/B Test）是验证方案有效性的黄金标准。无论是产品功能调整、排序策略升级，还是端到端推荐模型的更新，均需通过严谨的在线实验来评估其对用户体验与业务目标的真实影响。相比离线指标（如 AUC、NDCG）或回溯仿真（Backtest），A/B 测试能够捕捉用户在真实环境中的行为反馈，有效规避数据泄露、选择偏差和模型过拟合等问题，从而为决策提供可靠的因果证据。

一个标准的 A/B 实验包含若干核心组件：首先是 **Base 组（对照组）**，运行当前线上稳定版本的推荐策略，作

为效果对比的基准；其次是 **Exp 组（实验组）**，部署待验证的新策略，例如新模型、新特征或新召回通道；第三是 **分流机制（Traffic Allocation）**，通常通过对用户 ID、设备 ID 或请求 ID 进行哈希取模实现随机分流，确保各组用户在统计上同分布。实践中常采用如 95% Base : 5% Exp 的流量划分，并支持多层实验（Layered Experimentation）以提升流量复用效率；最后是 **实验指标（Online Metrics）**，分为核心业务指标（如人均观看时长、视频完播率、点赞/评论/分享率）与辅助诊断指标（如曝光多样性、冷门内容占比、负反馈率）。所有指标需在实验前明确定义，并绑定到统一的指标平台（Metric Binding），以确保计算口径一致。为保障结果可靠性，系统通常采用 **用户粒度聚合（User-level Aggregation）** 而非请求粒度，避免同一用户多次曝光带来的样本依赖；同时，实验周期应覆盖完整的用户行为周期（通常不少于 7 天），以消除周中/周末效应等时间偏差。

除基础 A/B 实验外，工业实践中还广泛采用多种进阶实验范式。例如，**Holdout 实验** 会保留一小部分流量（如 1%）长期不参与任何实验，作为全局基准组，用于监控系统整体趋势或进行跨实验横向比较；**Combo 实验**（多变量实验）可同时测试多个策略组合（如模型 A + 特征 X vs 模型 B + 特征 Y），适用于策略耦合场景，但需谨慎设计以避免交互效应干扰；**Backtest 实验**（伪在线回放）则利用历史请求日志，将新策略“回放”到过去流量上，虽不能替代真实 A/B 测试，但可作为上线前的低成本筛选手段。

A/B 测试的本质是因果推断，其结论的可信度依赖于若干关键前提：首先，**随机性** 必须得到保障，即分流过程不受用户属性（如新老用户、地域、设备类型）影响，避免组间系统性偏差；其次，**一致性（Sticky Assignment）** 要求同一用户在整个实验周期内始终归属同一实验组，防止策略切换干扰行为判断；第三，需尽量满足 **无干扰假设（SUTVA, Stable Unit Treatment Value Assumption）**，即一个用户的体验不应受其他用户实验分组影响——这在推荐场景中尤为关键，需警惕社交传播、热门效应或全局内容池变化带来的溢出影响。实验结果需通过统计检验（如 t 检验、Bootstrap 重采样或贝叶斯方法）判断指标变化是否显著，并计算最小可检测效应（MDE, Minimum Detectable Effect）以评估实验灵敏度。对于多指标场景，还需进行多重检验校正，以降低假阳性风险。

高质量的实验最终依赖于完善的日志体系与严格的数据质量保障。推荐系统通常设计两类核心日志：**曝光日志（Impression Log）** 记录每次推荐请求返回的视频列表、排序位置、实验分组信息（如 exp_id、group_id）等，用于构建样本和计算曝光相关指标；**行为日志（Action Log）** 则记录用户后续的点击、观看、点赞、评论、分享等反馈行为，并通过 req_id 与曝光日志精确关联，形成完整的“曝光-反馈”闭环。同时，工业界数据系统中也会建立数据质量监控机制，包括实验分流比例校验（确保实际流量与配置一致）、日志丢失率监控（如曝光-行为匹配率）、以及指标漂移告警（如 Base 组核心指标异常波动）。只有在数据完整、分流合规、指标稳定的前提下，A/B 实验的结论才具备真正的决策价值。

在实际工作中，算法工程师通常遵循一套标准化的实验演进流程。实验初期，会选取约 5% 的线上真实流量进行小范围测试，以快速验证新策略的初步效果。若在线指标（如人均观看时长、互动率）在统计上显著正向，且无明显负向副作用，则进入 **扩量阶段**，逐步提升实验流量比例（如扩至 10%、20% 甚至 50%），持续观察效果稳定性。当扩量后指标仍能在 5~7 天内保持稳健提升，且通过多维度诊断（如新老用户、不同内容类目）确认无局部失效，团队将执行 **实验推全（Full Rollout）**：将非 Holdout 流量全部切换至新策略，并将原实验参数设为系统默认值。为持续监控推全后的长期效果，工程师常会开启一个 **翻转 Backtest 实验（Flip Experiment）**，即交换原 Base 与 Exp 的参数配置，在 Holdout 或小流量上反向验证当前策略是否依然优于旧版本，从而形成闭环验证机制。

综上所述，一套科学、稳健的在线实验体系，不仅是推荐算法迭代的“试金石”，更是连接技术优化与业务增长的关键桥梁。它既要求严谨的因果推断框架与高质量的数据基础设施，也依赖于工程师对实验节奏、指标解读与风险控制的综合判断。

11.7 工程挑战

在传统推荐系统研究中，建模通常围绕用户-物品交互矩阵展开。早期方法如矩阵分解（Matrix Factorization）通过低秩近似捕捉用户与物品的潜在偏好；随着深度学习的发展，研究者进一步将交互矩阵作为输入，利用神经网络（如 Wide & Deep、Neural Collaborative Filtering 等）对用户-物品关系进行非线性拟合。然而，这类基于稠密或稀疏矩阵的建模范式在 **工业级大规模推荐系统** 中面临严峻的工程挑战——当系统需服务 **上亿级用户** 并面

对十亿级短视频内容库时，直接以矩阵形式存储用户-物品交互数据在存储、计算和扩展性上均不可行。

首先，在**离线数据存储层面**，全量用户-物品交互矩阵的规模将达到 $10^8 \times 10^9$ 量级，即使以稀疏格式存储，其内存与磁盘开销也远超实际工程承载能力。因此，工业实践中普遍采用**以用户为中心的聚合存储模式**：每个用户 ID 作为主键（key），其对应的值（value）是一个结构化的**用户行为序列**（User Action List），包含该用户历史上交互过的所有物品及其上下文信息，例如物品 ID、交互时间戳、所在页面位置、行为类型（点击、完播、点赞、评论、分享等）、观看时长、设备类型等。这种存储方式不仅大幅压缩了数据冗余，还天然支持序列建模与上下文感知的特征构造。这类行为序列已成为现代推荐模型（如 DIN、DIEN、SASRec 等）的核心输入特征，相关建模方法将在后续章节中详细展开。

其次，在**离线模型训练层面**，面对每日新增的数十亿级交互样本，单机训练框架完全无法满足时效性与吞吐需求。工业界普遍采用**分布式训练架构**，基于 Parameter Server 或 AllReduce 通信范式（如 TensorFlow PS、PyTorch DDP 等），将海量样本分片至多个计算节点并行处理。在此基础上，推荐架构工程师还需深度优化底层算子性能：例如针对稀疏 Embedding 查找进行缓存优化、对交叉特征进行高效组合、定制化开发适用于高维稀疏场景的优化器（如 AdaGrad 变种），甚至结合 GPU/TPU 加速稠密网络部分的计算。这些工程优化共同保障了模型能在合理时间内完成训练，并在效果与效率之间取得平衡。

第三，在**在线服务层面**，推荐系统必须在**数百毫秒内**完成从请求接收到最终结果返回的全过程，这对系统延迟提出了极高要求。考虑到候选物品池规模高达十亿级别，若对全库物品逐一调用精排模型打分，其计算量、网络传输开销与排序复杂度将呈指数级增长，完全不可接受。为此，工业推荐系统普遍采用**多阶段级联架构**（Multi-stage Cascade Architecture），以“漏斗”形式逐层筛选候选集：

- **召回阶段（Retrieval）**：通过多路策略（如协同过滤、向量近邻检索、图神经网络、语义匹配等）从十亿级物品库中快速筛选出数千至数万个候选物品；
- **粗排阶段（Pre-ranking）**：使用轻量级模型（如浅层 DNN 或蒸馏模型）对召回结果进行初步打分，选出 Top 100-500 个高质量候选；
- **精排阶段（Ranking）**：调用复杂深度模型（如多任务网络、序列建模模型）对粗排结果进行精细化打分与多目标融合；
- **重排阶段（Re-ranking）**：结合业务规则、多样性控制、上下文约束（如去重、打散、保量）对精排结果进行最终调整，输出 Top 5-10 个推荐结果。

该架构通过“**计算复杂度逐级递增、候选集规模逐级递减**”的设计，在保证推荐质量的同时，将整体响应时间控制在可接受范围内。

最后，**系统实时性**也是关键挑战之一。用户兴趣具有高度动态性，新行为（如刚点赞的视频）若不能及时反映到推荐结果中，将显著降低用户体验。为此，现代推荐系统普遍构建**实时特征管道**与**在线模型更新机制**：用户行为日志通过 Kafka 等高吞吐消息队列实时流入流处理引擎（如 Flink、Spark Streaming），生成分钟级甚至秒级更新的实时特征（如“最近 10 分钟点击序列”）；同时，模型参数也可通过增量更新或在线学习（Online Learning）方式，借助参数服务器或模型服务框架（如 TensorFlow Serving、TorchServe）实现热更新，确保推荐结果能快速响应用户最新意图。

综上所述，工业级推荐系统的构建远不止于算法模型本身，而是涵盖**数据存储、分布式训练、低延迟服务、实时反馈**四大核心工程维度的系统性工程。每一环节的优化都需在效果、效率与成本之间精细权衡，这也正是推荐系统从“实验室原型”走向“亿级用户产品”的关键所在。

11.8 本章小结

本章从工业实践视角出发，系统性地介绍了现代推荐系统的整体架构与核心要素。我们首先明确了推荐系统作为“用户与内容桥梁”的定位，并剖析了其在软件体系中的调用链路及响应约束；进而揭示了混合推荐系统如何在多业务并行的复杂生态中实现资源协调与目标平衡。通过对业务目标的拆解，我们看到不同场景（短视频、直播、电商、广告）对推荐系统的优化指标存在显著差异，这要求算法设计必须与业务语境深度耦合。

在系统实现层面，我们详细阐述了离线与在线架构的协同机制：离线侧依赖 Kafka、Redis、Flink 等构建高

可靠样本拼接与模型训练流水线；在线侧则通过多阶段级联架构（召回 → 粗排 → 精排 → 重排）在十亿级候选池中实现毫秒级响应。同时，日志与数据作为模型迭代的基石，其质量、丰富性与实时性直接决定了推荐效果的上限。而在线实验设计则为算法迭代提供了科学的因果验证框架，确保每一次策略更新都建立在可靠证据之上。

最后，我们归纳了推荐系统在工程落地中面临的四大核心挑战：超大规模数据的存储与建模、分布式训练的效率优化、低延迟服务的架构设计，以及实时反馈机制的构建。这些挑战共同构成了推荐系统从“可用”走向“好用”再到“智能”的关键门槛。

综上，推荐系统绝非单一模型或算法的堆砌，而是一个融合数据工程、分布式系统、机器学习与业务理解的复杂工程体系。后续章节将基于本章建立的宏观框架，逐层深入各核心模块（第 12 章）与关键模型（第 13 章），进一步揭示工业级推荐系统的实现细节与演进逻辑。

11.9 参考文献

- [1] Biye Jiang et al. “Dcaf: A dynamic computation allocation framework for online serving system”. In: *arXiv preprint arXiv:2006.09684* (2020).